

Scenario 4: ALB and Kubernetes Ingress Routing Failure

Objective

Trace HTTP 404 responses across Datadog, Route 53, ALB, Ingress, Service, endpoints, and pods.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/break-ingress.sh food-delivery
```

The script saves the current Ingress backend and changes the live route. Allow one or two minutes for the AWS Load Balancer Controller to reconcile before testing the public URL.

Situation

Users receive HTTP 404 from the food delivery home page even though the Kubernetes workloads report healthy.

Symptoms

- The public hostname returns HTTP 404.
- Kubernetes workloads remain Running and Ready.

What You Know

- `food-delivery.<lab-domain>` resolves and the ALB accepts connections.
- Pods are Running and Ready.
- Datadog Trace Explorer shows requests to `service:food-delivery-backend` with `http.status_code:404` and resource `GET /`.

Start With Datadog

Open **SRE Lab Scenario Signals**, then APM Trace Explorer. Filter `service:food-delivery-backend env:lab @http.status_code:404`. The **Unexpected backend root traffic** monitor uses `trace.express.request.hits.by_http_status` with `http.status_code:404` and the metric-normalized `resource_name:get_ /`. This is a backend APM signal, not ALB telemetry. Target health still requires AWS inspection.

Troubleshooting

Trace the Kubernetes portion of the request path.

```
kubectl get ingress -A
kubectl describe ingress food-delivery -n food-delivery
kubectl get svc -n food-delivery
kubectl describe svc food-delivery-frontend -n food-delivery
kubectl get endpoints -n food-delivery
kubectl get pods -n food-delivery
```

Then trace the AWS portion. Obtain ARNs from each preceding command.

```
aws elbv2 describe-load-balancers --region us-east-1
aws elbv2 describe-listeners --load-balancer-arn <alb-arn> --region us-east-1
aws elbv2 describe-rules --listener-arn <listener-arn> --region us-east-1
aws elbv2 describe-target-health --target-group-arn <target-group-arn> --region us-east-1
aws route53 list-resource-record-sets --hosted-zone-id <hosted-zone-id>
```

Important Clues

Follow DNS to ALB, listener, target group, Ingress, Service, endpoints, pod, and application. Determine why the backend answers a request intended for the frontend.

Root Cause

Record which routing hop sends traffic to the wrong destination. Confirm it with the instructor after the exercise.

Fix

```
./scripts/chaos/break-ingress.sh food-delivery --undo
kubectl describe ingress food-delivery -n food-delivery
```

The source-of-truth manifest is `ingress/food-delivery-ingress.yaml`, whose backend is `food-delivery-frontend:80`.

Validation

Test `https://food-delivery.<lab-domain>/`, confirm the controller reconciles the Ingress, and verify the backend `GET /` trace rate returns to zero in Datadog.

DevOps Lesson

Healthy pods do not guarantee healthy user traffic. Validate the whole routing path.

STAR Method

Situation

Summarize the user-visible 404 with healthy workloads.

Task

Explain the need to locate the failing routing hop.

Action

Describe the Datadog, AWS, and Kubernetes traffic-path checks.

Result

State how external access and trace recovery were verified.

Natural Spoken Version

Use the matching example in [DevOps STAR Scenarios](#) after completing the lab.